

AI-DRIVEN DEFECT DETECTION IN 3D PRINTING

Presented By Soumya Ranjan Senapati

Motivation for Project Work

-  **Prevents Catastrophic Failure:** Real-time detection of critical errors specifically spaghetti, blobs, stringing, under-extrusion, and warping stops the process before irreversible damage occurs to the printer's hotend or the structural integrity of the part is lost.
-  **Reduces Material Costs and Waste:** By automatically pausing or halting a failed job the moment a defect is identified, the system minimizes the consumption of wasted filament and reclaims valuable machine operating hours.
-  **Automates Quality Control:** Replaces subjective, intermittent manual inspection with continuous, real-time computer vision monitoring, ensuring consistent, high-precision manufacturing yield without human fatigue.

Objectives of Project Work

- **To compile a robust, multi-source dataset:** Aggregate and standardize image datasets from Roboflow to create a comprehensive and diverse visual repository of five specific FDM 3D printing defects (blobs, spaghetti, stringing, under-extrusion, and warping).
- **To train and optimize a real-time detection model:** Implement and fine-tune the YOLOv26m object detection architecture on the custom dataset to accurately localize and classify printing anomalies.
- **To evaluate model performance for manufacturing environments:** Assess the model's precision, recall, and inference speed (mAP) to ensure it is viable for real-time, in-situ monitoring without lagging behind the printer's physical speed.
- **To establish a foundation for automated quality control:** Develop a vision-based framework that can eventually be integrated with printer firmware to automatically pause or stop failed prints, reducing material waste and machine downtime.

Methodology - Dataset Aggregation & Analytics

- **Base Dataset Compilation:** Merged multiple diverse datasets to establish a robust foundation of 5,874 raw images.
- **High Annotation Density:** The base dataset features 22,028 bounding box annotations across the 5 target classes, averaging 3.8 defects per image.
- **Defect Typology:** The dataset captures five distinct morphological failures such as Blobs (excess material accumulation), Spaghetti (tangled filament from complete adhesion loss), Stringing (fine oozing during travel moves), Under Extrusion (gaps from insufficient flow), and Warping (corners detaching due to thermal contraction).
- **Class Distribution:** Annotations reflect real-world defect frequencies, heavily weighted towards Blobs (13,206) and Spaghetti (4,994).
- **Background / Null Examples:** Strategically retained 1,551 healthy print images to train the model on normal operations and minimize false positive detections.

Methodology - Dataset Aggregation & Analytics

Number of Images

5874

23 missing annotations
1551 null examples

Number of Annotations

22028

3.8 per image (average)
Across 5 classes

Average Image Size

0.41 mp

from 0.01 mp
to 49.94 mp

Median Image Ratio

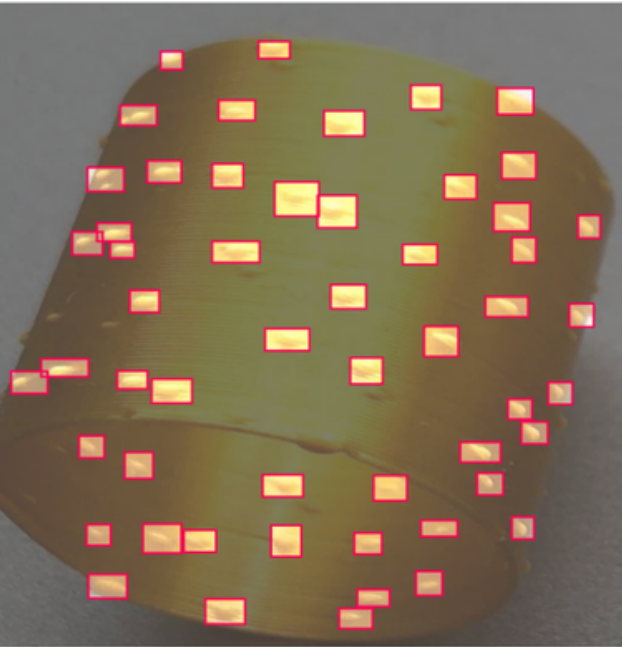
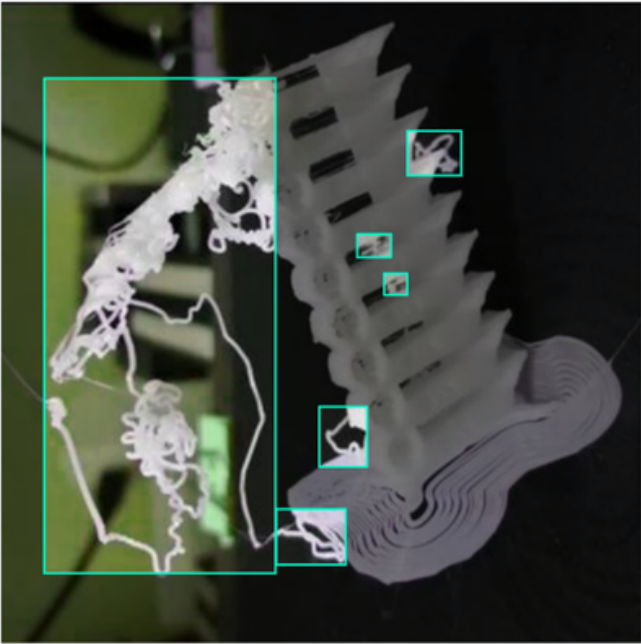
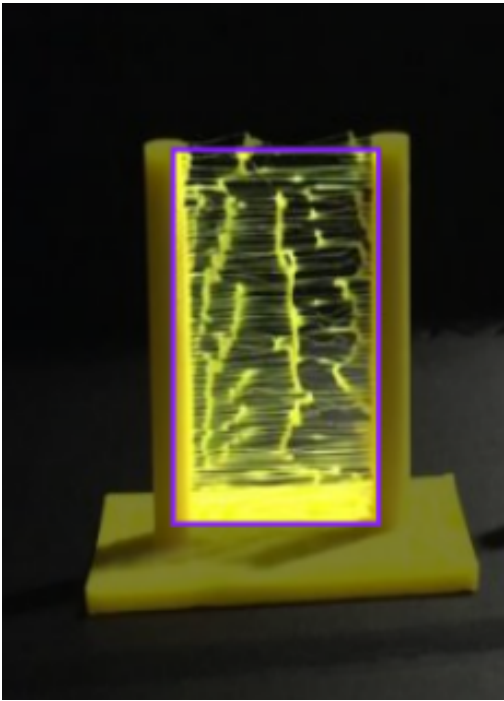
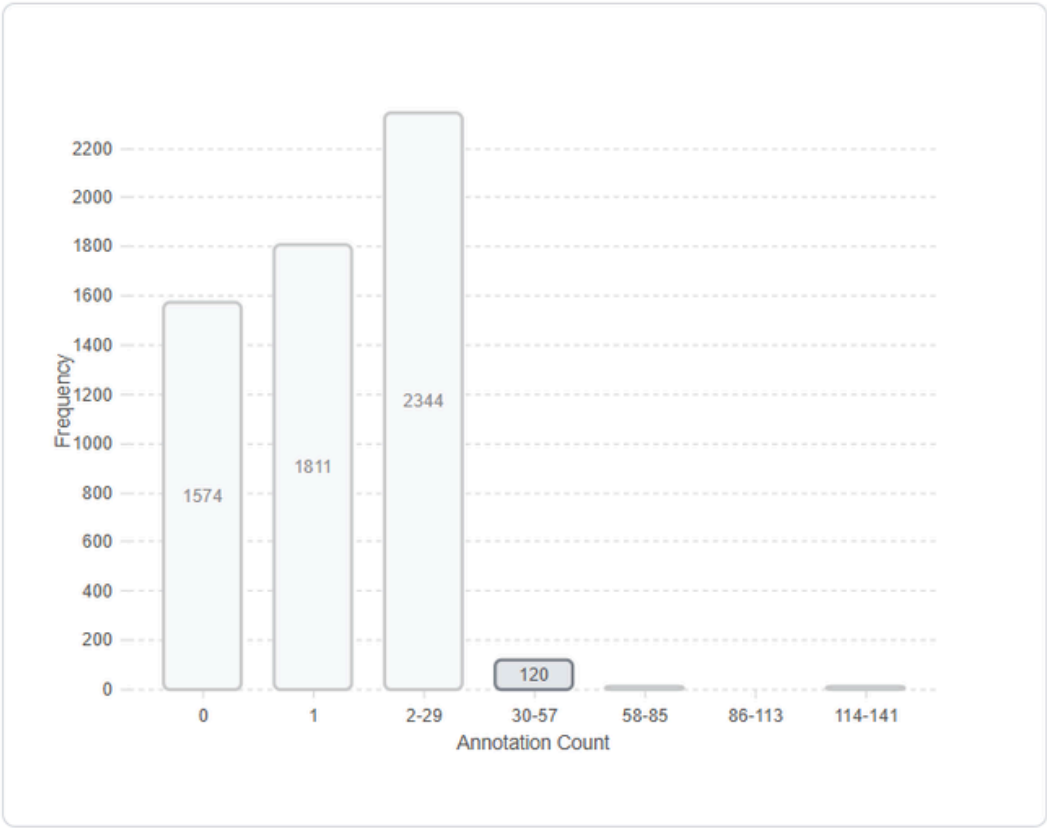
640x640

square

COLOR ?	CLASS NAME	COUNT ↻
	Blobs	13,206
	Spaghetti	4,994
	Stringing	1,979
	Under Extrusion	1,055
	Warping	794

Histogram of Object Count by Image ⓘ

Overview of how many classes are annotated in each image in your dataset.



Methodology - Data Splitting, Preprocessing & Augmentation

- **Initial Data Split:** Divided the base 5,874 images (100%) into 4,347 for training (~75%) and 1,527 for validation (~25%) to ensure a robust evaluation baseline.
- **Global Preprocessing (All Data):** Standardized both Train and Valid sets by applying Auto-Orient and resizing to 640x640 (Fit with reflected edges) to maintain physical aspect ratios without introducing artificial black borders.
- **Targeted Augmentation (Train Only):** Applied image augmentations exclusively to the training set to prevent data leakage and ensure the validation set strictly tests real-world, unmodified conditions.
- **Techniques Applied:** Utilized Horizontal Flip, Rotation (-10° to $+10^{\circ}$), Brightness (Between -20% and +20%), Blur (Up to 1px), and Noise (Up to 1.53% of pixels) to simulate diverse factory lighting, camera vibration, and hardware limitations.
- **Final Dataset Volume:** Generating 2 outputs per training example (2X) expanded the training set from 4,347 to exactly 8,694 images. The validation set remained at 1,527 images, yielding a final dataset of 10,219 total images ready for model training.

Methodology - Model Training

- **Algorithm Selection:** Deployed the YOLOv26m (You Only Look Once, medium variant) architecture. This model was selected for its optimal balance of high detection accuracy (mAP) and low-latency inference, which is critical for real-time monitoring of rapid FDM toolhead movements.
- **Computing Environment:** Bypassed local hardware limitations by utilizing Kaggle's cloud computing platform. Leveraged free GPU accelerators (e.g., NVIDIA P100/T4) to handle the heavy computational load of training on a 10,219-image dataset.
- **Implementation Pipeline:** Exported the annotated and augmented dataset from Roboflow in YOLO PyTorch format and executed the training script using the Ultralytics framework over the specified number of epochs.

```
from roboflow import Roboflow
rf = Roboflow(api_key="my-api-key")
project = rf.workspace("ml-ssuvd")
        |.project("3d-printing-defect-detection-evopu")
version = project.version(2)
dataset = version.download("yolo26")
```

```
from ultralytics import YOLO

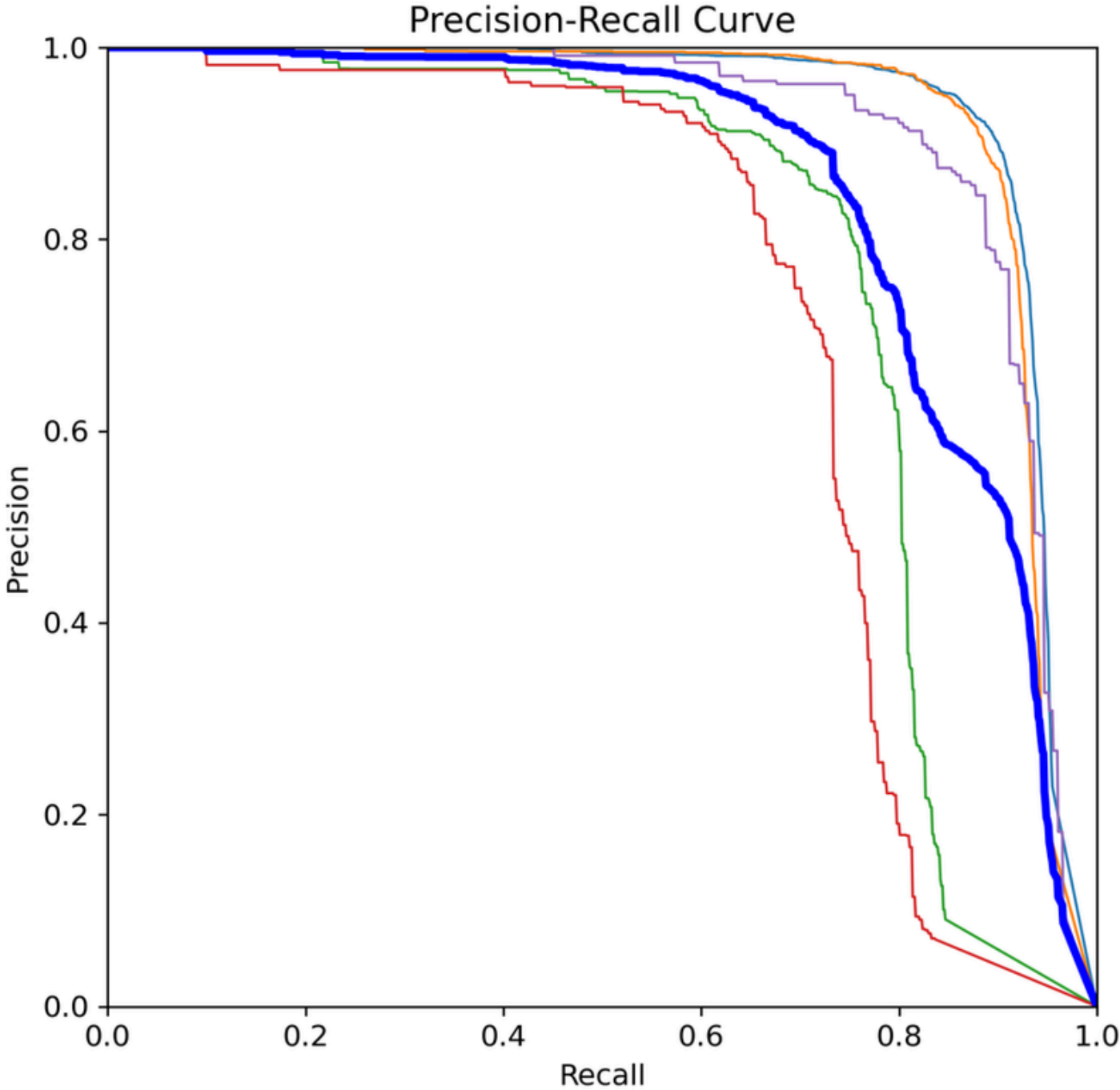
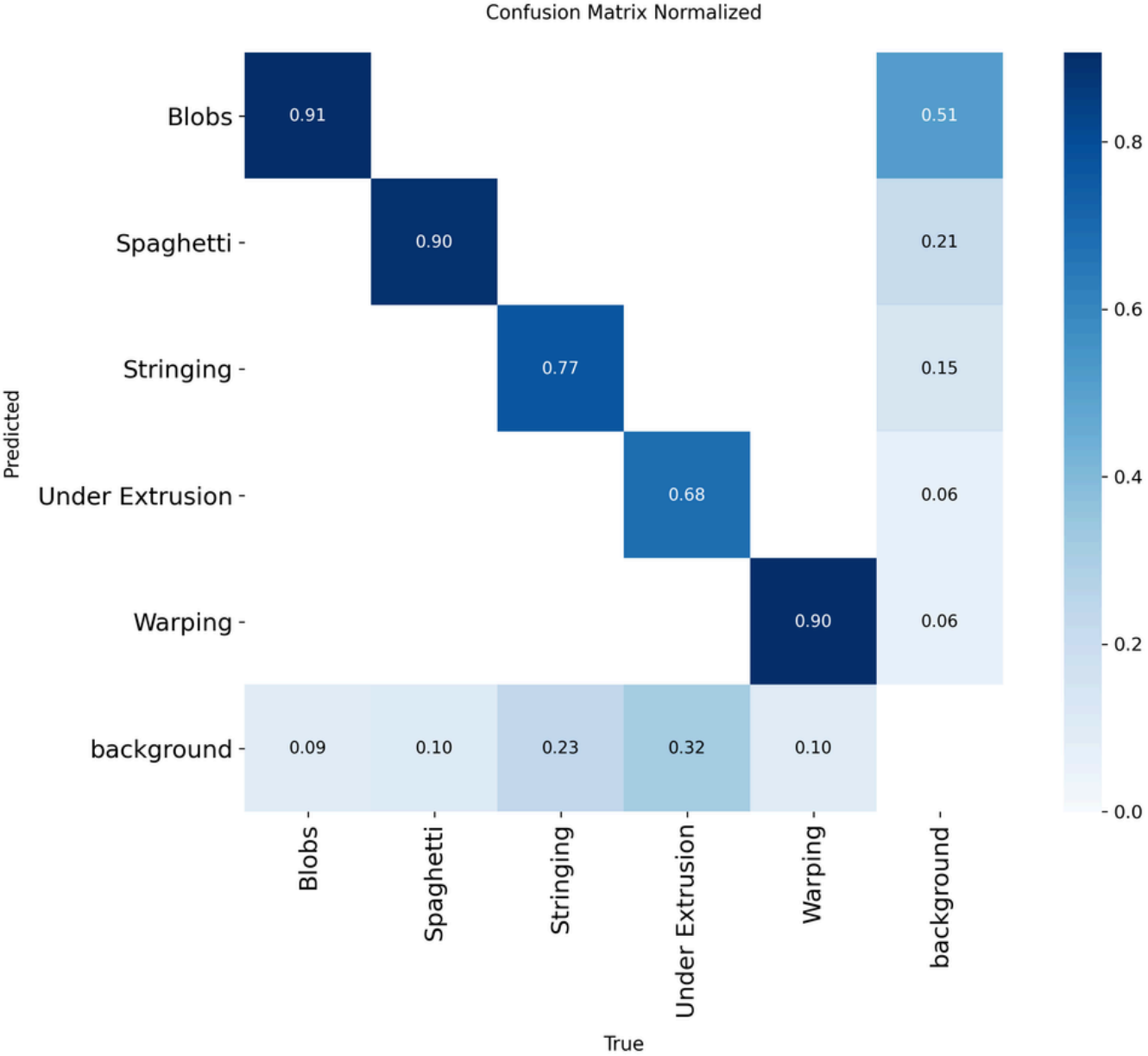
model = YOLO("yolo26m.pt")

results = model.train(
    data=f"/kaggle/working/3D-Printing-Defect-Detection-2/data.yaml",
    epochs=100,
    imgsz=640,
    batch=32,
    device="0,1",
    patience=10,
    cache=False,
    save_period=10,
    project="/kaggle/working/training",
    name="model_V1"
)
```

Results

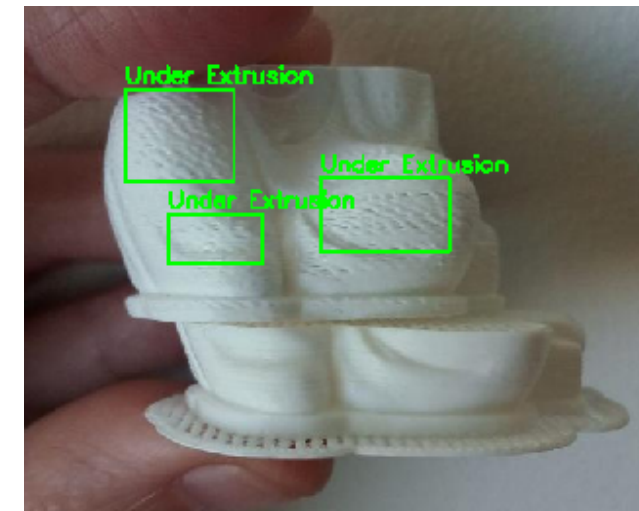
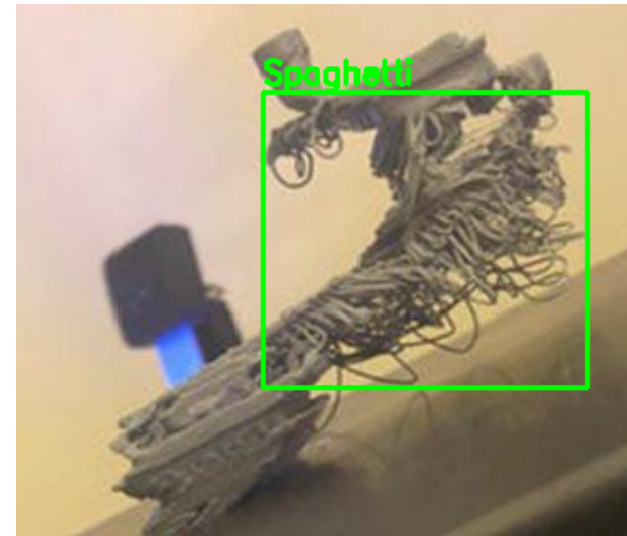
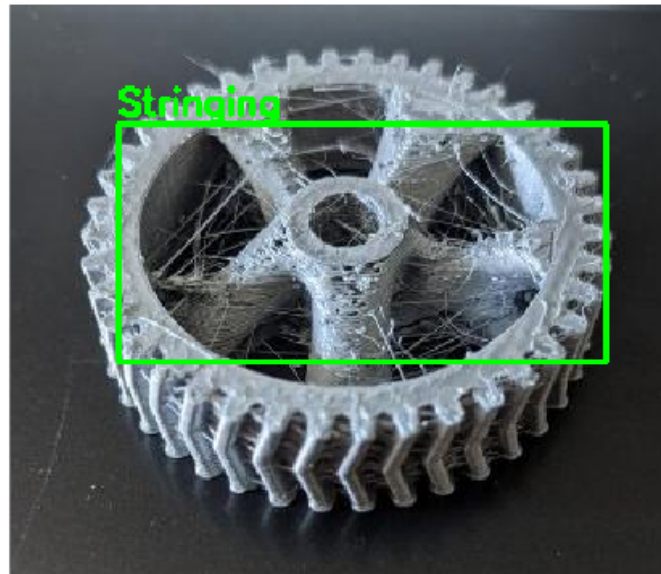
- **Training Convergence:** The YOLOv26m model was successfully trained over 60 epochs, demonstrating stable learning. Both training and validation losses decreased consistently, indicating the model did not overfit to the training data.
- **High Precision (88.44%):** Achieved a peak Precision of 0.8844 by the final epoch. In a manufacturing context, this means that when the AI flags a defect (like spaghetti or a blob), it is correct nearly 90% of the time, minimizing unnecessary printer pauses or false alarms.
- **Defect Identification / Recall (79.90%):** Reached a Recall rate of 0.7990. This demonstrates the system's capability to successfully catch and localize roughly 80% of all actual printing anomalies present in the unseen validation set.
- **Overall Accuracy (mAP):** Attained a strong mean Average Precision (mAP@50) of 85.58% across the defect classes, proving the model's reliability. Furthermore, the stringent mAP@50-95 metric reached 62.67%, highlighting that the AI draws tight, accurate bounding boxes directly around the physical defects.

Results

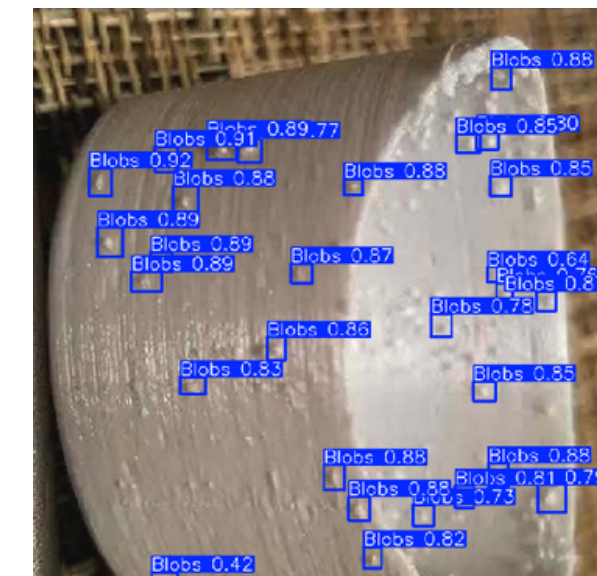
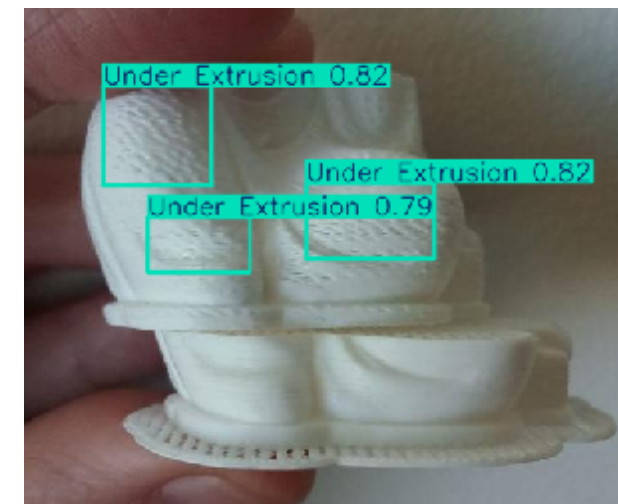
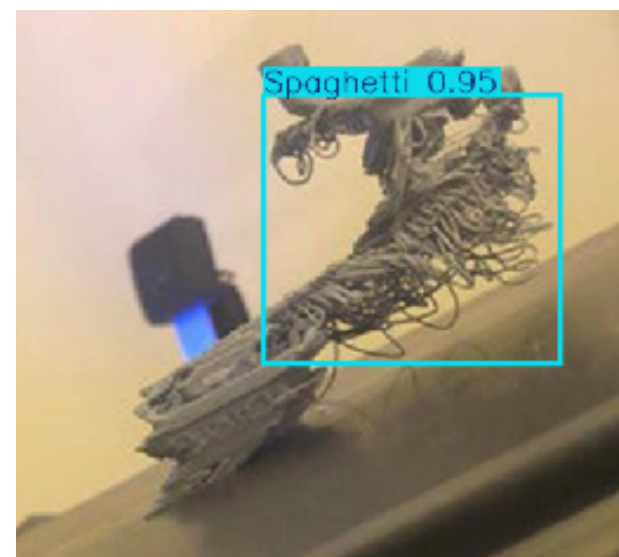


Results

Actual Labels



Model Predictions



Future Work & Project Scope

- **Real-Time Detection Setup:** Establish a live video feed of the 3D printer's build plate using either the laptop's built-in camera or an external USB webcam connected directly to the laptop running the YOLOv26m model.
- **Serial Communication & Integration:** Develop a Python-based control script to establish serial communication between the laptop and the 3D printer's mainboard.
- **Automated Print Halting (Closed-Loop Control):** Program the system to automatically trigger a "Pause Print" or "Emergency Stop" G-code command the moment the AI detects a critical defect (e.g., spaghetti or severe blob) above a set confidence threshold.
- **Latency and Reliability Testing:** Conduct live physical test prints with intentionally induced errors to measure the system's reaction time and ensure the laptop can process frames and halt the printer before irreversible damage occurs.

Thank You

For your attention

